

ECEN 248: Introduction to Digital Design
Department of Electrical and Computer Engineering
Texas A&M University

Laboratory Exercise #11
The Traffic Light Controller Lab

Lab exercise created and tested by
Abbas Fairouz, He Zhou, Joshua Mashburn, and Sunil P. Khatri

Revised by Alexandra Chiu in April 2026



1 Introduction

The lab this week will continue the introduction to FSMs with a simple Traffic Light Controller design project. In the pre-lab, you will have the opportunity to create a state diagram of a *Mealy* machine, which implements a traffic light controller, based on provided guidelines. You will then use this state diagram to write the behavioral Verilog description of the traffic light controller. The testing of your traffic light controller will take place during the lab session using the Vivado development tools and the ZYBO Z7-10 board. We will make use of the LEDs on the ZYBO Z7-10 board to display the outputs of our digital circuit.

2 Background

Background information necessary for the completion of this lab assignment will be presented in the next few subsection. The pre-lab assignment that follows will test your understanding of the background material.

2.1 The Mealy Machine

As mentioned in the previous lab, the *Mealy* machine is a Finite State Machine (FSM) in which the outputs of the machine are dependent not only on the state of the machine but also on the input to the machine. As a reminder, Figure 1 highlights the distinction between the *Mealy* and the Moore machine with a dotted line that connects the input to the next-state logic.

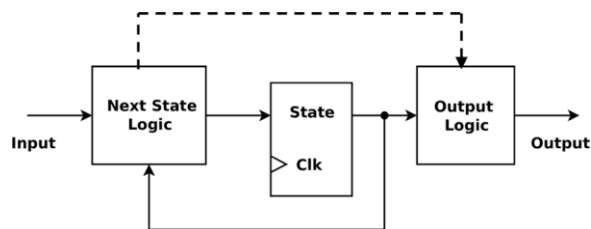


Figure 1: The Mealy Machine

As an example, consider the 2-bit saturating counter discussed previously. Let us suppose that we were asked to add an additional output signal, *Sat*, that is asserted when the counter is saturated in either the maximum or minimum state. Assuming that the counter is not considered to be in saturation when it first enters the minimum or maximum state, but rather when it remains in either of those states, we would need to use the state and the input of the machine to create this output signal. In terms of the state diagram, *Sat*

will need to be based on the state transitions of the machine as seen in Figure 2. Notice that *Sat* is defined on the transitions (i.e. edges of the graph) rather than on the states (i.e. nodes of the graph).

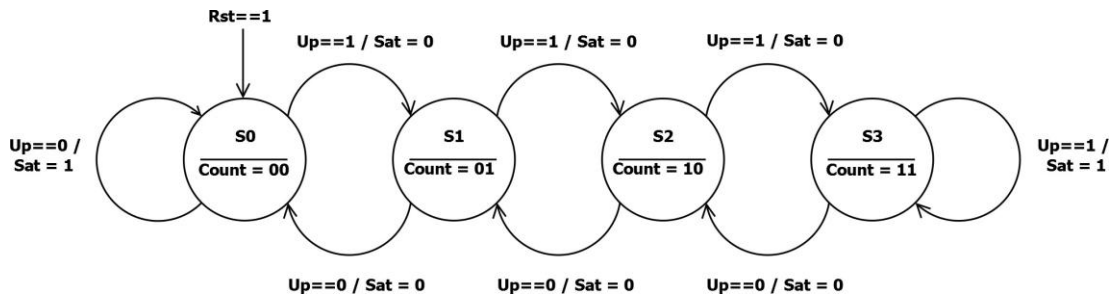


Figure 2: 2-bit Saturating Counter State Diagram

The behavioral Verilog for the modified 2-bit saturating counter is nearly identical to the previous description with the exception of an additional **always** block, which describes the combinational logic used to generate the *Sat* signal. To make clear the dependencies *Sat* has on the state and input, the **always@(state or Up)** clause has been used. It should be noted that **always@(*)** would have worked as well.

```

1  `timescale 1ns / 2ns
2  `default_nettype none
3
4  /* This is a behavioral Verilog description of a
5     2-bit saturating counter */
6
7  module saturating_counter(Count, Sat, Up, Clk, Rst);
8
9     output wire [1:0] Count; // 2-bit output
10    output reg Sat; // driven in an always block
11    input wire Up; // input bit asserted for up
12    input wire Clk, Rst; // the usual inputs to a synchronous circuit
13
14    /* we haven't talked about parameters much but as you can see,
15       they make our code much more readable! */
16    parameter S0 = 2'b00,
17              S1 = 2'b01,
18              S2 = 2'b10,
19              S3 = 2'b11;
20
21    /* intermediate nets */
22    reg [1:0] state; // 4 states require 2 bits
23    reg [1:0] nextState; // will be driven in always block!
24
25    /* describe Sat output logic */
26    /* Notice that it closely resembles the next-state logic */
27    always @(state or Up) begin // purely combinational!
28        case(state)
29            S0: begin
30                if(Up) Sat = 1'b0; // not saturated

```

```

31         else Sat = 1'b1; // saturated
32     end
33     S1: Sat = 1'b0;
34     S2: Sat = 1'b0;
35     S3: begin
36         if(Up) Sat = 1'b1; // saturated
37         else Sat = 1'b0; // not saturated
38     end
39 endcase
40 end
41
42 /* Describe next state logic */
43 always@(*) // purely combinational!
44     case(state)
45         S0: begin
46             if(Up) // count up
47                 nextState = S1;
48             else // saturate
49                 nextState = S0;
50         end
51         S1: begin
52             if(Up) // count up
53                 nextState = S2;
54             else // Count down
55                 nextState = S0;
56         end
57         S2: begin
58             if(Up) // count up
59                 nextState = S3;
60             else // count down
61                 nextState = S1;
62         end
63         S3: begin
64             if(Up) // saturate
65                 nextState = S3;
66             else // count down
67                 nextState = S2;
68         end
69     endcase
70
71 /* now describe synchronous logic to hold our state! */
72 always@(posedge Clk) // only update state on the clock edge!
73     if(Rst) // reset state
74         state <= S0;
75     else
76         state <= nextState;
77
78
79 /* describe output logic, which in this case happens to just be wires */
80 assign Count = state;
81
82 endmodule

```

2.2 Generating Timing Delays

In lab, you will need to implement pauses between state transitions in order to mimic the operation of a real traffic light. As you may have guessed, counters can be used to generate precise timing delays or pauses. Figure 3 depicts this concept with a state machine that after *Start* is asserted, waits in state *S1* for 1000 clock cycles and then in state *S2* for 2000 cycles. To accomplish this, the output signal from the state machine, *RstCount*, resets the counter, and in turn, the output of the counter (and input to the state machine), *Count*, controls the transitions from states *S1* and *S2*.

The actual time delay for each state can be easily calculated by multiplying the delay in terms of clock cycles by the clock period. For example, if the clock driving the aforementioned counter has a 50 MHz frequency, then the wait times in state *S1* and *S2* are given by:

$$t_{s1} = \frac{10^3}{50 * 10^6} = 20 * 10^{-6} = 20\mu s$$

$$t_{s2} = \frac{2 * 10^3}{50 * 10^6} = 40 * 10^{-6} = 40\mu s$$

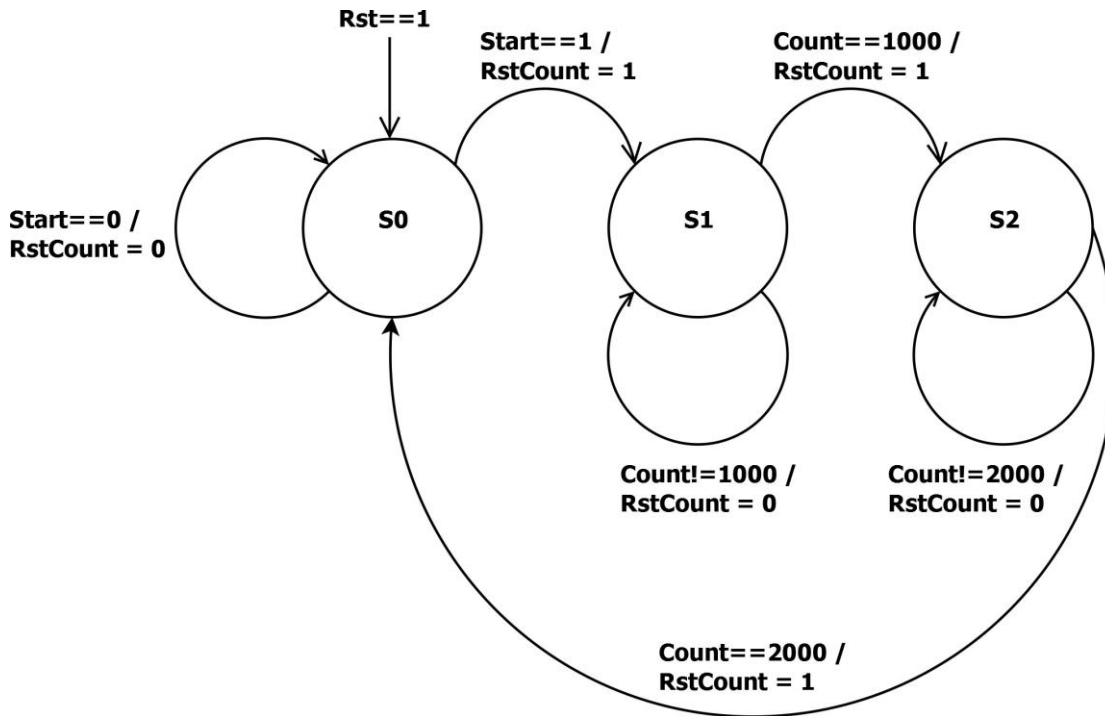


Figure 3: Counter Controlled Paused Transitions

3 Pre-lab

The objective of the pre-lab assignment is to create the state diagram and write the Verilog for the FSM that you will load onto the ZYBO Z7-10 board during lab. Therefore, the following subsections will describe the design we are attempting to create in detail. The pre-lab deliverables at the end will state what it is you are required to have prior to attending your lab session.

3.1 The Traffic Light Controller

This week you will design a traffic light controller for the highway and farm road intersection shown in Figure 4. For simplicity, we will assume that the traffic lights on opposite ends of the intersection are the same; in other words, if the light for the North bound traffic is green, then the light for the South bound traffic is also green. The traffic light controller outputs two 2-bit signals, *highwaySignal* and *farmSignal*, for

the highway road and the farm road, respectively. The following encoding is used for both signals:

11: *green*
 10: *yellow*
 01: *red*

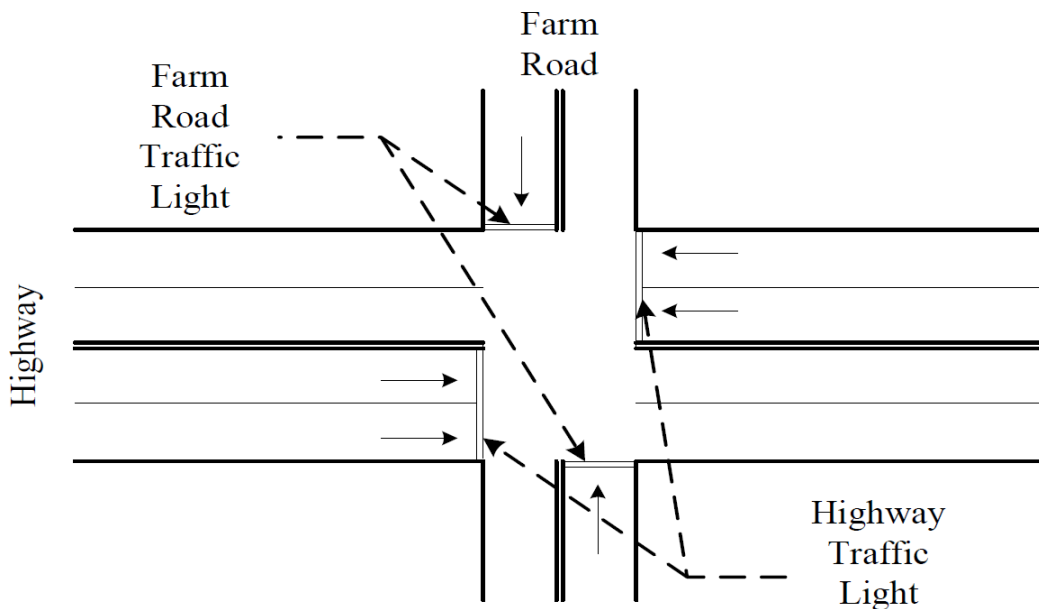


Figure 4: Highway and Farm Road Crossing

The traffic light controller will be made up of three main components as shown in Figure 5. You will be responsible for describing in Verilog the counter and FSM.

Obviously, to avoid accidents, the highway lights and farm road lights must not be green at the same time. Furthermore, the transitions from green on each road must be followed by 3 seconds of yellow and a 1 second period in which all directions are red. The latter allows motorists to clear the intersection prior to switching the direction of traffic. Because the traffic on the highway is much greater than that of the farm road, the traffic light must remain green on the highway longer than on the farm road. For this design, the highway light must remain green for 30 seconds, while the farm road light must remain green for only 15 seconds. Table 1 summarizes the above guidelines. Notice the partially blank column on the far right of Table 1 with the delays in terms of clock cycles. The entries in this column represent the values the counter

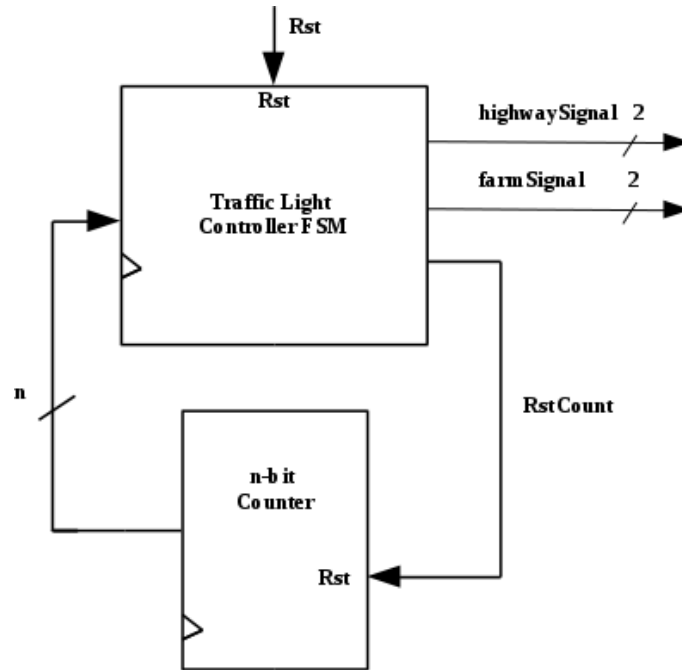


Figure 5: The Initial Traffic Light Controller Circuit

must reach in each state before transitioning to the next state. You will need to complete this column assuming a 50 MHz clock.

The brain of the traffic light controller is the FSM. For the the initial design, *Count* and *Rst* constitute the only input to the FSM. The output of the FSM includes the *highwaySignal* and *farmSignal* previously mentioned as well as *RstCount*, the counter reset. The operation of the FSM can be described by Table 1 and the time line in Figure 6. We can also describe this behavior using a state diagram similar to that seen in Figure 3. One of the pre-lab exercises will require you to do just that.

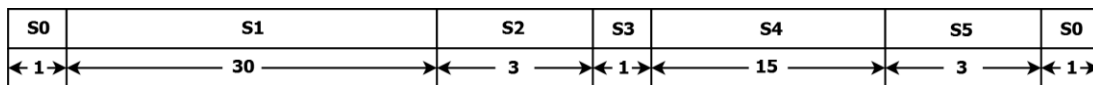


Figure 6: Time Line in Seconds of Initial FSM design

Table 1: Traffic Light Controller States

State	Highway Output	Farm Road Output	Delay (seconds)	Delay (cc)
S0	red	red	1	50,000,000
S1	green	red	30	
S2	yellow	red	3	
S3	red	red	1	
S4	red	green	15	
S5	red	yellow	3	

3.2 Pre-lab Deliverables

Include the following items in your pre-lab submission.

1. Fill in the remaining entries in Table 1. You may find the discussion on *Generating Timing Delays* in the background section helpful.
2. Based on the column entries you just calculated, what is the necessary value of n in Figure 5?
3. Given Table 1 and Figure 6, create a state diagram for the traffic light controller FSM. Be sure to include the appropriate input and output labels. Assume $S0$ is the reset state.
4. Now describe the traffic light controller FSM in Verilog using the following module interface:

```

1 module tlc_fsm(
2     output reg [2:0] state, //output for debugging
3     output reg RstCount // use an always block
4     output reg [1:0] highwaySignal, farmSignal,
5     input wire [n:0] Count, //use n computed earlier
6     input wire Clk, Rst //clock and reset
7 );

```

4 Lab Procedure

Experiment 1 involves the implementation and testing of the design presented in the pre-lab, while experiment 2 requires that you extend the functionality of the pre-lab design.

4.1 Experiment Part 1

In experiment 1, you will synthesize the traffic light controller with the FSM you designed in the prelab. Correct functionality of the design will be determined by loading the design onto the ZYBO Z7-10 board and verifying that it operates per per-lab specifications.

1. Synthesize and Implement the traffic light controller discussed in the pre-lab using Vivado.
 - (a) Create a new Vivado project called 'lab11' and add the Verilog code you wrote in the pre-lab to the new project.
 - (b) Create a source file called 'tlc_controller.v' and use the Verilog code below as a starting point for describing the circuit in Figure 5.

```

1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  /*This module describes the top level traffic light controller (version 1) module discussed in Lab 11*/
5
6  module tlc_controller_ver1(
7      output wire [1:0] highwaySignal, farmSignal, // connected to LEDs, 4-bits in total
8      output wire [3:0] JB, //output state for debugging!
9      input wire Clk,
10     input wire Rst // use a reset, the buttons will provide input to our top level circuit
11 );
12
13     /* intermediate nets */
14     wire RstSync; // reset after it's been synchronized
15     wire RstCount; // signal obtained from the fsm
16     reg [30:0] Count;
17
18     assign JB[3] = RstCount; //use the highest of our debugging bits for Reset Count
19
20     /* synchronize button inputs */
21     synchronizer SYNC(RstSync, Rst, Clk);
22
23     /* instantiate FSM */
24     tlc_fsm FSM(
25         .state(JB[2:0]), // wire states up to JB for debugging
26         //finish things up here...
27         .Rst(RstSync)
28     );
29
30     /* counter with asynchronous reset. Notice how reset sync can trigger the always block! */
31     always @(posedge Clk or posedge RstSync) begin
32         if (RstSync || RstCount) begin
33             Count <= 31'b0;
34         end
35         else begin
36             Count <= Count + 1;
37         end
38     end
39 endmodule
40
```

- (c) Copy the following source files from the course directory into your 'lab11' directory.
 - 'synchronizer.v', the synchronizer module for our asynchronous inputs
 - 'tlc_controller.xdc', the top-level XDC
 - (d) Add the aforementioned source files to your Vivado project
 - (e) Generate a bitstream for the design, fixing all errors and warnings.
2. Load the design onto the FPGA board and test its functionality.
 - (a) Program the board using the bit stream created in the previous step.
 - (b) Press the 'BTN0' button to reset the design and observe the output on the LEDs.
 - (c) The LEDs, LED3(D18) and LED2(G14), indicate Highway Signal, whereas Farm Signal is indicated by LED1(M15) and LED0(M14)
 - (d) Verify that your traffic light controller cycles through the states appropriately.

DEMO: If your design works properly, **demonstrate your progress to the TA;**

- (e) If it does not work, you may want to use the logic analyzer for debugging. Additionally, you may find the FSM test bench file, 'tlc_fsm_tb.v', to be helpful. Since simulation is quite slow, you will want to reduce the delays in your FSM module by several orders of magnitude prior to simulation.

NOTE: The provided test bench does not perform automatic verification so you will need to examine the waveform output to determine correct operation.

4.2 Experiment Part 2

The traffic light controller designed in the pre-lab is fairly inefficient. Most of the time, there are no cars on the farm road, except of course when there is an Aggie football game. Therefore, a sensor has been installed that asserts the signal, *farmSensor*, when a car is present on the farm road in either direction. In this experiment, you will modify your current design to support the new sensor.

1. Use the steps below to integrate the sensor signal into your current design.
 - (a) Add an input port, *farmSensor*, to your FSM module.
 - (b) Modify the FSM with the following the specifications:
 - The highway light shall remain GREEN as long as the sensor signal is LOW but must be GREEN for at least 30 seconds at a time.
 - Once transitioned to GREEN, the farm road light shall remain GREEN for at least 3 seconds.
 - As long as the sensor signal is HIGH, the farm road light shall remain GREEN but not for more than 15 additional seconds in order to give priority to the highway.

- All signal light transitions from GREEN shall proceed to YELLOW and then to RED as before.

HINT: You can achieve the desired behavior fairly trivially by inserting new states. For example, if the light is supposed to remain GREEN for as long as the sensor is low, but must be GREEN for at least 30 seconds, you can keep a GREEN state that waits 30 seconds, and instead of transitioning to yellow, transitions to a new states that waits for the farmSensor.

- (c) Modify the 'tlc_controller.v' source file and the corresponding XDC file to support the new signal. Use the 'BTN1' button on the ZYBO Z7-10 board to mimic the sensor output. Don't forget to properly synchronize the input signal, as was done with Rst.
2. Load the design onto the ZYBO Z7-10 board and ensure proper operation.
 - (a) Create a bitstream for programming the FPGA. If warnings or errors are generated in this step, they are more than likely associated with your XDC file modifications.
 - (b) Program the ZYBO Z7-10 board with the bit stream created in the previous step.

DEMO: Demonstrate your progress to the TA once you have determined that your modified design is operating per specifications.

5 Post-lab Deliverables

Please include the following items in your post-lab write-up.

1. Include the source code with comments for **all** modules that you wrote or modified in lab. You do **not** need to include code that was provided! Remember that code without comments will not be accepted!
2. Include the state diagram for the modified traffic light controller FSM.
3. **(Honors)** Explain why we did not have to de-bounce the sensor signal, *farmSensor*, even though it was generated by a push-button for our prototyped design.

6 Important Student Feedback

The last part of the lab requests your feedback. We are continually trying to improve the laboratory exercises to enhance your learning experience, and we are unable to do so without your feedback. Please include the following post-lab deliverables in your lab write-up.

Note: If you have any other comments regarding the lab that you wish to bring to your instructor's attention, please feel free to include them as well.

1. What did you like most about the lab assignment and why? What did you like least about it and why?
2. Were there any sections of the lab manual that were unclear? If so, what was unclear? Do you have any suggestions for improving the clarity?

3. What suggestions do you have to improve the overall lab assignment?